

```
-----  
-----  
-- Engineer: Olasubomi Borishade  
--  
-- Create Date: 06/11/2026  
-- Design Name: Processing Element Test Bench  
-- Module Name: Processing_Element_Test_Bench - Behavioral  
-- Project Name: Parameterized Matrix Multiply Engine  
--  
-- Description: Test bench for the processing element which  
--               wraps a MAC unit and forwards Input_A rightward and  
--               Input_B downward to neighboring processing  
elements,  
--               with one register stage per direction.  
--  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
-----
```

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.NUMERIC_STD.ALL;  
use STD.ENV.FINISH;
```

```
entity Processing_Element_Test_Bench is  
end Processing_Element_Test_Bench;
```

```
architecture Behavioral of Processing_Element_Test_Bench is
```

```
Constant Input_Widths      : INTEGER := 8;  
Constant Accumulator_Widths : INTEGER := 32;  
Constant Clock_Period      : TIME    := 10 ns;
```

```

Signal Input_As      : SIGNED (Input_Widths - 1 downto 0) :=
(others => '0');

Signal Input_Bs      : SIGNED (Input_Widths - 1 downto 0) :=
(others => '0');


Signal Clocks        : STD_LOGIC := '0';
Signal Resets        : STD_LOGIC := '0';
Signal Enables        : STD_LOGIC := '0';
Signal Clear_Accumulators : STD_LOGIC := '0';


Signal Output_As      : SIGNED (Input_Widths - 1 downto 0);
Signal Output_Bs      : SIGNED (Input_Widths - 1 downto 0);
Signal Outputs        : SIGNED (Accumulator_Widths - 1 downto
0);


begin

PE_Inst: entity work.Processing_Element(Behavioral)
    Generic map (
        Input_Width      => Input_Widths,
        Accumulator_Width => Accumulator_Widths
    )

    Port map (
        Input_A      => Input_As,
        Input_B      => Input_Bs,
        Clock         => Clocks,
        Reset         => Resets,
        Enable        => Enables,
        Clear_Accumulator => Clear_Accumulators,
        Output_A      => Output_As,
        Output_B      => Output_Bs,
        Output        => Outputs
    );


    Clock_process : process
    begin
        Clocks <= '0';
        wait for Clock_Period / 2;
    end process;
end;

```







```

    assert (Output_As = to_signed(3, Input_Widths))
        report "TEST 4a FAILED: Expected Output_A 3, got " &
integer'image(to_integer(Output_As))
    severity error;

    assert (Output_Bs = to_signed(4, Input_Widths))
        report "TEST 4b FAILED: Expected Output_B 4, got " &
integer'image(to_integer(Output_Bs))
    severity error;

    assert (Outputs = to_signed(12, Input_Widths))
        report "TEST 4c FAILED: Expected Output 12, got " &
integer'image(to_integer(Outputs))
    severity error;

    Clear_Accumulators <= '1';
    wait for Clock_Period;

    Clear_Accumulators <= '0';
    Input_As      <= to_signed(4, Input_Widths);
    Input_Bs      <= to_signed(5, Input_Widths);
    wait for Clock_Period;

    assert (Output_As = to_signed(4, Input_Widths))
        report "TEST 4d FAILED: Expected Output_A 5, got " &
integer'image(to_integer(Output_As))
    severity error;

    assert (Output_Bs = to_signed(5, Input_Widths))
        report "TEST 4e FAILED: Expected Output_B 6, got " &
integer'image(to_integer(Output_Bs))
    severity error;

    assert (Outputs = to_signed(20, Input_Widths))
        report "TEST 4f FAILED: Expected Output 42, got " &

```

```

integer'image(to_integer(Outputs))
    severity error;

Input_As    <= to_signed(6, Input_Widths);
Input_Bs    <= to_signed(7, Input_Widths);
wait for Clock_Period;

assert (Output_As = to_signed(6, Input_Widths))
    report "TEST 4g FAILED: Expected Output_A 7, got " &
integer'image(to_integer(Output_As))
    severity error;

assert (Output_Bs = to_signed(7, Input_Widths))
    report "TEST 4h FAILED: Expected Output_B 8, got " &
integer'image(to_integer(Output_Bs))
    severity error;

assert (Outputs = to_signed(62, Accumulator_Widths))
    report "TEST 4i FAILED: Expected Outputs 98, got " &
integer'image(to_integer(Outputs))
    severity error;

Input_As    <= to_signed(8, Input_Widths);
Input_Bs    <= to_signed(9, Input_Widths);
wait for Clock_Period;

assert (Output_As = to_signed(8, Input_Widths))
    report "TEST 4j FAILED: Expected Output_A 8, got " &
integer'image(to_integer(Output_As))
    severity error;

assert (Output_Bs = to_signed(9, Input_Widths))
    report "TEST 4k FAILED: Expected Output_B 9, got " &

```

```
integer'image(to_integer(Output_Bs))  
    severity error;  
  
    assert (Outputs = to_signed(134, Accumulator_Widths))  
        report "TEST 4l FAILED: Expected Outputs 134, got " &  
integer'image(to_integer(Outputs))  
    severity error;  
  
Input_As      <= to_signed(0, Input_Widths);  
Input_Bs      <= to_signed(0, Input_Widths);  
Enables       <= '0';  
wait for Clock_Period;  
  
-----  
--                                                     DONE  
                                     --  
-----  
-----  
  
report "All Processing Element tests completed."  
severity note;  
  
finish;  
end process;
```

end Behavioral;